Farmacia Hospitalaria

Guion de la defensa

Sistema de Gestión de Stock — Farmacia Hospitalaria

Presentación, demostración en vivo y preguntas probables del tribunal.

Trabajo Final Integrador — Tecnicatura Universitaria en Programación — UTN
Noviembre de 2026

Índice

1. Diapositivas (≈ 10 minutos) 3

2. Demostración en vivo (≈ 8 minutos) 4

- Preparación (el día anterior y 30 minutos antes) 4
- Recorrido 4
- Si algo falla 5

3. Preguntas probables y respuestas 5

- Sobre el problema y el alcance 5
- Sobre la base de datos 5
- Sobre la concurrencia y el stock 5
- Sobre la seguridad 6
- Sobre el diseño del código 6
- Sobre las pruebas 7
- Sobre el frontend 7
- Sobre el presupuesto 7
- Preguntas "trampa" 7

Guion de la defensa

Duración sugerida: **20 minutos** de presentación + demostración, y después preguntas. [COMPLETAR: ajustar al tiempo que indique la cátedra.]

1. Diapositivas (≈ 10 minutos)

Una idea por diapositiva, poco texto, la explicación la decís vos.

#	Diapositiva	Qué decir (resumen)	Tiempo
1	**Título**: Sistema de Gestión de Stock para la Farmacia de un Hospital Público. Nombre, carrera, fecha.	Presentarse.	0:30
2	**El problema**: medicamentos que vencen, áreas que piden todos los días, control en planillas.	"Si un medicamento vence en el depósito se pierde plata pública; si falta, se afecta a un paciente. Hay que saber de qué lote, cuándo vence y a dónde fue cada unidad."	1:00
3	**El prototipo anterior**: los 8 problemas (tabla corta).	"Partí de un prototipo con problemas graves de seguridad y de datos. Ninguno se repite y cada uno tiene una prueba automática que lo demuestra."	1:00
4	**Objetivo y alcance**: qué hace y qué no (no hay datos de pacientes).	Objetivo general en una frase.	0:45
5	**Arquitectura**: diagrama navegador → React → API Spring Boot → PostgreSQL, todo en Docker.	"Cliente-servidor. El frontend solo muestra; las reglas y la seguridad están en el backend."	1:00
6	**Modelo de datos**: diagrama ER de 9 tablas.	"Normalizado hasta 3FN. No guardo nada que se pueda calcular: el stock total es la suma de los lotes no vencidos. Sin borrados en cascada: las bajas son lógicas."	1:15
7	**FEFO**: ejemplo con dos lotes (100 que vence en diciembre, 200 que vence en marzo; piden 150 → 100 + 50).	"La persona no elige el lote: el sistema saca del que vence primero."	1:00
8	**Concurrencia**: dos farmacéuticos piden 60 de un lote de 100 al mismo tiempo.	"Sin control, los dos leen 100 y el stock queda en −20. Uso un bloqueo en la base ordenado por id: uno pasa, el otro recibe 'stock insuficiente'. Lo demuestra una prueba que lanza dos hilos a la vez."	1:15
9	**Seguridad**: BCrypt, JWT, 3 roles y 6 permisos, permisos leídos de la base en cada pedido.	"Si el admin desactiva a alguien, pierde el acceso en su próxima acción, aunque su token no haya vencido."	1:00
10	**Calidad**: 100 pruebas en el backend (con PostgreSQL real en Docker), 101 en el frontend, accesibilidad automática.	"Cada cambio se verifica en segundos."	0:45
11	**Mejora y presupuesto**: órdenes de compra; 444 h + 128 h a $25.000 = $14.300.000.	Explicar cómo se calculó el valor hora.	0:45
12	**Conclusiones** y "¿Preguntas?"	Tres aprendizajes: proteger las reglas en varias capas, la concurrencia no es un caso raro, las fechas y zonas horarias.	0:45

Diagramas listos para copiar: informe final (sección 8) y carpeta del proyecto (archivos 03 y 04). Exportalos como imagen desde https://mermaid.live.

2. Demostración en vivo (≈ 8 minutos)

Preparación (el día anterior y 30 minutos antes)

1. `docker compose down -v` y `docker compose up --build` con `SPRING_PROFILES_ACTIVE=demo` (datos frescos, con fechas relativas a ese día).
2. `cd frontend && npm run dev`.
3. Abrir el navegador en http://localhost:5173 con zoom al 100 % (o 110 % si el proyector es chico).
4. Tener abiertas tres pestañas o ventanas: `admin`, `farmaceutico` y `consulta`. Como la sesión está en `sessionStorage`, cada pestaña tiene su propia sesión. **Abrí cada usuario en una pestaña nueva escribiendo la dirección, no duplicando la pestaña.**
5. Tener las capturas de `docs/capturas/` a mano por si algo falla (plan B).
6. Tener Swagger abierto en otra pestaña: http://localhost:8080/swagger-ui.html.

Recorrido

#	Acción	Qué mostrar / decir
1	Ingresar como `farmaceutico`.	Menú según rol. Panel con los cuatro indicadores.
2	Clic en el indicador **Vencidos**.	Lleva a la lista filtrada. "Vencido es antes de hoy; el día del vencimiento todavía se usa."
3	**Dar de baja** un lote vencido con motivo.	Queda en 0 y desaparece de la lista.
4	**Ingreso de mercadería**: un lote nuevo de Paracetamol que vence en 10 días.	Aviso "vence pronto, podés registrarlo igual". Mensaje de éxito con el stock resultante.
5	**Nueva transferencia** a Guardia: Paracetamol por una cantidad mayor que el primer lote.	Columna Disponible / Queda. "Revisar y confirmar" → "Confirmar transferencia".
6	Mostrar el **comprobante**.	Dos renglones del mismo medicamento con lotes distintos: **FEFO**. Imprimir (vista previa) con lugar para firmas.
7	Nueva transferencia pidiendo más de lo disponible.	"Disponible: X (pediste Y)" y no se descontó nada: **todo o nada**.
8	Detalle del medicamento.	Lotes en orden FEFO y movimientos: ingreso, egreso, baja. **Trazabilidad**.
9	**Reportes** → Transferencias → Excel y PDF.	Abrir el Excel: columnas con ancho correcto, fechas reales, filtros.
10	Pestaña de `consulta`.	No ve botones de alta/baja. Escribir `/usuarios` en la URL → "No tenés permiso".
11	Pestaña de `admin` → **Usuarios** → desactivar a `farmaceutico`. Volver a la pestaña de `farmaceutico` y hacer cualquier acción.	Lo saca al ingreso: "los permisos se leen de la base en cada pedido". Reactivarlo después.
12 (si sobra tiempo)	Swagger: `POST /api/transferencias` sin token.	401 en JSON con `codigo`.

Si algo falla

- **No levanta Docker:** mostrar las capturas y el código; explicar qué se vería.
- **"No se pudo conectar con el servidor":** el backend no está levantado → `docker compose ps`.
- **No hay lotes vencidos en la demo:** los datos demo los crean relativos a hoy; si se borraron, se muestra la pantalla de lotes "Por vencer".
- No improvises cambios de código durante la defensa.

3. Preguntas probables y respuestas

Sobre el problema y el alcance

¿Por qué no guardan datos de pacientes? Porque las transferencias van a áreas, no a personas. Guardar pacientes implicaría datos sensibles de salud (Ley 25.326 de Protección de Datos Personales y Ley 26.529 de Derechos del Paciente) y otro alcance. Quedó explícitamente fuera.

¿Qué es FEFO y por qué no FIFO? FEFO es "primero vence, primero sale". FIFO es "primero entra, primero sale". Con medicamentos puede entrar después un lote que vence antes; con FIFO ese lote se quedaría en el estante y se vencería.

¿Por qué no se pueden anular transferencias? Para que el historial sea confiable: lo que se registró no se borra. Si hubo un error o una devolución, se corrige con un ajuste que queda registrado con usuario, fecha y motivo.

Sobre la base de datos

¿Cómo justifican la tercera forma normal? Ningún atributo no clave depende de otro no clave. Por ejemplo, el lote no guarda el nombre del medicamento (dependería de `medicamento_id`); el stock total no se guarda porque depende de los lotes; el número `TR-000123` se deriva del id. La única redundancia es `saldo_resultante` en los movimientos, que es un dato histórico de auditoría que nunca se actualiza, así que no genera anomalías.

¿Por qué una tabla `detalle_transferencia`? Resuelve la relación muchos a muchos entre transferencias y lotes: una transferencia toma varios lotes y un lote se reparte en varias transferencias. Además registra cuánto salió de cada lote.

¿Por qué claves primarias numéricas y no el código del medicamento o el CUIT? Son claves sustitutas: si un código se cargó mal, se corrige sin romper las relaciones. La clave natural igual está protegida con `UNIQUE`.

¿Qué pasa si borro un proveedor? No se puede borrar: no hay `DELETE` en la API y las claves foráneas no tienen cascada. Se desactiva; sus lotes siguen y su historial también. El prototipo borraba en cascada y se perdía todo.

¿Qué es Flyway? Una herramienta que aplica los scripts SQL de la base en orden (`V1`, `V2`, `V3`) y registra en una tabla cuáles ya aplicó. Así la base de cualquier entorno queda igual, y nunca se edita un script ya aplicado: los cambios van en uno nuevo.

Sobre la concurrencia y el stock

¿Cómo garantizan que el stock no quede negativo? En tres niveles: la validación de la entrada (cantidades positivas), la entidad `Lote` (no deja descontar más de lo que hay) y un `CHECK` en la base (`cantidad_actual >= 0`), que es la última barrera. Y para pedidos simultáneos, el bloqueo pesimista.

¿Qué es un bloqueo pesimista? ¿Por qué no optimista? El pesimista reserva las filas (`SELECT … FOR UPDATE`) hasta que termina la transacción; el otro pedido espera. El optimista deja avanzar a los dos y falla al guardar si alguien cambió el dato, y hay que reintentar. Con stock escaso y pedidos sobre los mismos lotes, el optimista fallaría seguido.

¿Por qué ordenan por id al bloquear? Para evitar el deadlock: si una transferencia bloquea A y después B, y otra bloquea B y después A, se quedan esperando la una a la otra. Si todas bloquean en el mismo orden (por id), eso no puede pasar. Hay una prueba con dos medicamentos pedidos en orden inverso.

¿Cómo lo probaron? `TransferenciaConcurrenciaIT`: un lote de 100, dos hilos que arrancan juntos (con un `CountDownLatch`) y piden 60 cada uno contra un PostgreSQL real. Resultado: uno 201, otro 409, stock final 40.

Sobre la seguridad

¿Cómo se guardan las contraseñas? Con BCrypt, factor 12. Es un hash irreversible, con sal aleatoria y lento a propósito: no se puede "desencriptar" ni probar millones por segundo.

¿Qué es un JWT y qué tiene adentro el de ustedes? Un token firmado que el servidor entrega al iniciar sesión y el navegador manda en cada pedido. El nuestro solo tiene el emisor, el id del usuario y las fechas de emisión y vencimiento. No lleva el rol: los permisos se leen de la base en cada pedido.

Si el token tiene solo el id, ¿no lo puedo cambiar por otro id? No: está firmado con una clave secreta (HMAC-SHA256). Si se cambia un carácter, la firma deja de coincidir y el servidor responde 401.

¿Dónde está la clave secreta? En una variable de entorno (`JWT_SECRET`), nunca en el código. Si falta o tiene menos de 32 bytes, la aplicación no arranca. Lo prueba `ConfiguracionTest`.

¿Por qué el token está en `sessionStorage` y no en una cookie? Se borra al cerrar la pestaña (bueno para PC compartidas). Una cookie HttpOnly sería más resistente a XSS, pero obligaría a protegerse de CSRF y a cambiar el backend. El riesgo de XSS se mitiga con React (escapa todo), la prohibición de `innerHTML` y una CSP estricta.

¿Por qué CSRF está deshabilitado? CSRF aprovecha que el navegador manda las cookies solo. Nuestra API no usa cookies: el token va en una cabecera que el navegador no agrega por su cuenta. Entonces CSRF no aplica.

¿Qué diferencia hay entre 401 y 403? 401: no sé quién sos (sin token, token inválido o vencido, usuario desactivado). 403: sé quién sos pero no tenés permiso.

Si oculto los botones en el frontend, ¿no alcanza? No. Ocultar es comodidad; la seguridad está en el backend. Cada endpoint tiene `@PreAuthorize`. Aunque alguien llame a la API a mano, recibe 403.

¿Por qué el login responde igual si el usuario no existe? Para no revelar qué usuarios existen. Incluso el tiempo de respuesta es parecido, porque se calcula un hash igual.

Sobre el diseño del código

¿Por qué capas? Cada una tiene una responsabilidad: el controlador traduce HTTP, el servicio aplica reglas, el repositorio accede a datos. Se puede cambiar una sin tocar las otras y se prueban por separado.

¿Qué es un DTO y por qué no devuelven las entidades? Un objeto solo con los datos que entran o salen por la API. Si devolviera la entidad `Usuario`, saldría el hash de la contraseña (era el problema 4 del prototipo).

¿Qué es la inyección de dependencias? En lugar de que una clase cree sus dependencias (`new LoteRepository()`), las recibe en el constructor y Spring se las pasa. Permite reemplazarlas por simulaciones en las pruebas.

¿Por qué usan un `Clock` en lugar de `LocalDate.now()`? Por dos motivos: la zona horaria (el contenedor está en UTC y a las 21 h ya sería mañana) y las pruebas (con un reloj fijo pruebo "vence hoy" o "venció ayer" siempre igual).

¿Por qué el stock se calcula con SQL y no en Java? Para no traer todos los lotes a memoria: la base suma y agrupa mucho más rápido, y así se puede filtrar y ordenar por stock con paginación.

Sobre las pruebas

¿Qué diferencia hay entre una prueba unitaria y una de integración? La unitaria prueba una clase aislada, reemplazando lo demás con simulaciones (Mockito); es rápida. La de integración prueba varias partes juntas: acá, la API completa contra un PostgreSQL real en Docker (Testcontainers).

¿Por qué una base real en las pruebas y no una en memoria (H2)? Porque quiero probar lo que hace PostgreSQL: los `CHECK`, el `FOR UPDATE`, el SQL específico. Con otra base podría pasar la prueba y fallar en producción.

¿Qué es TDD? ¿Lo usaron? Escribir primero la prueba que falla y después el código que la hace pasar. Se usó en la lógica crítica: FEFO, alertas, CUIT y el lote.

Sobre el frontend

¿Por qué React y no HTML con JavaScript? Por los componentes reutilizables (tabla, modal, insignia) y por TypeScript: los tipos se generan del contrato del backend, así si cambia la API el frontend no compila.

¿Qué hace TanStack Query? Maneja los datos del servidor: los guarda en caché, muestra carga y error, y después de una operación marca como desactualizado lo que cambió para volver a pedirlo.

¿Cómo manejan la accesibilidad? Contraste AA verificado, todo con teclado, foco visible, etiquetas en los campos, estados con palabra y no solo color, modales nativos, y una prueba automática con axe en todas las pantallas.

Sobre el presupuesto

¿Cómo calcularon el valor hora? Sueldo mediano semi senior en pesos según Sysarmy ($2.500.000) / 160 horas ≈ $15.600, por 1,6 por ser independiente (sin aguinaldo ni vacaciones, con monotributo y equipo propio) ≈ $25.000.

¿Hay costos de licencias? No: todo es software libre (Java, Spring, PostgreSQL, React, Docker, OpenPDF, fastexcel).

Preguntas "trampa"

¿Qué mejorarías si tuvieras más tiempo? Órdenes de compra (la mejora presupuestada), envío real de correos, renovación del token, una prueba de integración específica para bajas y ajustes, y sumar el frontend al `docker-compose`.

¿Qué fue lo más difícil? La concurrencia y las fechas. [COMPLETAR con la experiencia personal.]

¿Usaste inteligencia artificial? [COMPLETAR con honestidad según cómo se trabajó.] Si se usó: explicar que fue una herramienta, que cada parte se especificó con criterios de aceptación, se revisó y se verificó con pruebas automáticas, y que podés explicar cada decisión (para eso está la carpeta `estudio/`).